CodeSport

Sandya Mannarswamy

Welcome to this month's CodeSport, where we will continue our discussion on algorithms to support mutual exclusion, a fundamental necessity in concurrent programming.

Our last month's takeaway question was for readers to come up with a mutual exclusion algorithm (which is deadlock-free) for two threads, by combining the *SimpleLock1* and *SimpleLock2* algorithms that we discussed in last month's column. The idea is to combine the '*flag*' array from *SimpleLock1* with the '*sacrificer*' variable in *SimpleLock2*.

Recall that in *SimpleLock1*, each thread sets the *flag* array element corresponding to its ID as 1, to indicate that it is interested in entering the critical section. However, it deadlocks if both the threads run concurrently.

On the other hand, in *SimpleLock2*, each thread sets itself up as the victim, and lets the other thread execute first. Therefore, if the two threads do not run concurrently, then one thread can end up indefinitely waiting for the other thread to reset the victim attribute. As we noted in last month's column, the conditions under which *SimpleLock1* and *SimpleLock2* end up deadlocked are mutually complementary. Therefore, combining the two algorithms should avoid deadlock.

Petersen's mutual exclusion algorithm

Since none of the replies I received for this problem were fully correct, I'll give the correct solution: We use both a *flag* array, and a *victim* variable. Each thread sets the *flag* array element corresponding to its ID, to indicate that it is interested in entering the critical section. It also sets the victim to itself, giving preference to the other thread to enter the critical section. However, a thread waits only if both the *flag* array element for the other thread and the *victim* is set to this thread.

```
boolean flag[2];
int victim;

void CombinedLock()
{
     int i = pthread_self();
     int j = i - 1;

    // set the flag array element to indicate that it is interested in
entering critical section
flag[i] = true;

 // Give preference to other thread
     victim = i;

     while (flag[j] && (victim == i)){}
}
void CombinedUnlock()
{
        int i = pthread_self();
        flag[i] = false;
}
```

Consider the situations that caused deadlocks in the *SimpleLock1* and *SimpleLock2* algorithms. If both the threads run concurrently with their *flag* array element set to 1, then based on the value of the *victim* variable, one of the threads will enter the critical section. If the two threads do not run concurrently, then the running thread will see that the flag array element corresponding to the non-running thread is not set, and hence it can enter the critical section. Thus, with the *CombinedLock* algorithm, the deadlock situation is avoided in both cases. This algorithm is also known as Petersen's mutual exclusion algorithm. Now, can you show whether Petersen's algorithm is starvation-free?

Impact of memory consistency model on mutual exclusion algorithms

In our discussion on mutual exclusion algorithms, we have not strictly proved that an algorithm is correct by ensuring mutual exclusion always. I leave it to the reader to prove that our mutual exclusion algorithms are, in fact, correct. You can use the 'proof by contradiction' method to do this. Consider the *CombinedLock* algorithm; its correctness depends on the fact that (a) the setting of the *flag* array element happens before the setting of the *victim* variable, and (b) the setting of the *victim* variable by a thread should occur before it reads the *flag[other_thread]* array element, as part of evaluating the *while* loop condition. Now what would happen if either of these assumptions were violated? I leave it to the reader to show that violation of the program order for these memory location updates can lead to both the threads entering the critical section simultaneously, thereby violating the mutual exclusion.

By requiring that the instructions occur in program order for each thread, we have made an inherent assumption about the underlying memory model. We have assumed that the underlying memory consistency model is sequentially consistent. Before we see what is meant by a sequentially consistent memory model, let us first understand the term